- الف - پدیده چه الزامات و شرایطی را دارد، بسته به نوع موضوع باید تشریح شود که چه هدفی از مدلسازی دنبال میشود.
- ب - فرضیات حاکم بر پدیده چیست؟ متغیرها و پارامترهای مختلف چه تاثیری بر تابع هدف دارند؟
- ج - مدل با چه ابزاری در ریاضیات قابل تحلیل و بررسی است؟ ممکن است شما به آن ابزار تسلط نداشته باشید، لازم است کمی راجع به آن توضیح دهید.
- د - نتیجه گیری، یعنی مدل نوشته شده چه نتایجی را در اختیار ما میگذارد و ارزش توجه به آنرا بیان کنید. به بیان دیگر مزیتهای مدل چیست.

## مدلسازی پروازهای عبوری از فضای ایران

---

| نیازمندی ها | |
| --- | --- |
| رکورد پروازهای عبوری | **داده های پروازی** |
| نرخ پروازهای عبوری بر حسب مسیر | **اطلاعات صورتحساب** |
| برچسب زمان ورود | **تاریخ پروازها** |
| کد یکتا ایکائو | **شناسه شرکت های هواپیمایی** |

| | نیازمندی ها |
|---|---|
| **تاریخچه داده ها** 📈 | بازه زمانی کافی برای محاسبه الگو |

| | شرایط و ملاحظات |
|---|---|
| 🛫 **الگوی پروازی** | فصلی، چارتری، برنامه، غیر برنامه ای |
| 🔄 **تغییرات مسیرهای هوایی** | بسته بودن مسیر، تداخل مسیرها یا راه های هوایی جدید |
| 🌍 **مخاطرات تأثیرات جغرافیایی** | تحریم،مناطق جنگی یا روابط دیپلماتیک |
| 📉 **حساسیت نرخ** | ارزیابی کشش ← دوری ایرلاین ها از فضای هوایی گران |
| 💾 **کیفیت اطلاعات** | برچسب های زمانی نادقیق یا اطلاعات مالی اشتباه |

| هدف | مزیت |
|---|---|
| دسته بندی شرکت های هواپیمایی | تمرکز بر مشتری های بالقوه |
| پیش بینی درآمدهای عبوری | برنامه ریزی بودجه و زیرساخت |
| تحلیل تاثیرات سیاست گذاری | تنظیم نرخ بر اساس رفتار |
| بهینه سازی نرخ هوایی | ایجاد تعادل بین درآمد و ترافیک |

---

## 🛍️ Up-Selling, Cross-Selling, and Repeat Purchases

A reduction in customer churn directly uplifts the revenue. With customer behavior modeling, businesses can easily look at customer segments that are ripe for up-selling, cross-selling, and repeat purchases. These three sales maximizing tactics will maximize CLTV and also bring in more dollars to the business coffers.

| Concept | Definition | Example | Goal |
|---|---|---|---|
| **Up-Selling** | Selling a **more expensive or upgraded** version of the product the customer is considering. | Customer wants a basic phone → you suggest a premium model with better features. | Increase the **order value**. |
| **Cross-Selling** | Selling **related or complementary** products to what the customer is buying. | Customer buys a laptop → you suggest a laptop bag or mouse. | Increase the **basket size**. |
| **Repeat Purchase** | Customer **returns to buy again** after a previous purchase. | Customer buys shoes → comes back next month to buy another pair. | Build **loyalty** and **lifetime value**. |

---

## 📈 Concept Relationships

$$\text{Up-Selling:} \quad \text{Encourage buying a higher-tier product } (P^+)$$

$$\text{Cross-Selling:} \quad \text{Encourage buying complementary products } (P + Q)$$

$$\text{Repeat Purchase:} \quad \text{Customer returns for } P \text{ after time } t$$

---

## 🧠 Quick Mnemonics

- Up-Sell ↔ Upgrade the product
- Cross-Sell ↔ Add related products

- Repeat Purchase ↔ Retain and resell over time

---

# 📋 Agenda

- CLV

  تخمین سود آتی به ازای هر مشتری با استفاده از نرخ حفظ مشتری و نرخ تنزیل
- FBR

  تمرکز بر بهینه سازی درآمدهای حوزه هوانوردی
- RFM

  بخش بندی مشتریان بر اساس الگو های رفتاری گذشته

| Model | Variable / Parameter | Symbol | Description |
|---|---|---|---|
| **CLV** *(Customer Lifetime Value)* | Margin per period | $m$ | سود کسب شده به ازای هر مشتری در واحد زمان |
| | Retention rate | $r$ | احتمال ادامه رابطه مشتری در دوره بعدی |
| | Discount rate | $d$ | ارزش فعلی جریان‌های نقدی در هر دوره |
| | CLV formula | $CLV$ | $CLV = \frac{m \cdot r}{1-r+d}$ |
| | Lifetime revenue projection | – | پیش‌بینی سود کل از یک مشتری |
| **FBR** *(Flight-Based Revenue)* | Number of overflights | $N$ | تعداد کل پروازهای عبوری از حریم هوایی |
| | Distance per flight | $D$ | میانگین مسافت طی شده در حریم هوایی |
| | Overflight fee rate | $R$ | هزینه به ازای هر ناتیکال مایل |
| | Elasticity of demand | $E_d$ | حساسیت پروازها به تغییرات $R$ (کشش) |
| | Scenario adjustment factors | – | برای مدل‌سازی تغییرات جغرافیایی، فصلی یا رقابتی استفاده می‌شود |
| | Total revenue | $TR$ | $TR = N \times D \times R$ |
| **RFM** *(Recency–Frequency–Monetary)* | Recency | $R$ | زمان سپری شده از آخرین فعالیت مشتری (smaller = more recent) |
| | Frequency | $F$ | تعداد فعالیت مشتری (e.g., purchases, flights) |
| | Monetary | $M$ | کل مبلغی که مشتری پرداخت کرده است |
| | RFM score | – | رتبه‌بندی ترکیبی یا چندک R، F، M |
| | Segmentation | – | گروه‌بندی مشتریان (e.g., champions, at risk, lost) |
| | Combined into a single number | – | $\mathrm{RFM}_i = 100 \cdot R_i^* + 10 \cdot F_i^* + M_i^*$ |

---

## Customer Lifetime Value (CLV) — Mathematical Definitions

The Customer Lifetime Value (CLV) formula calculates the total revenue a business can expect from a single customer over the entire duration of their relationship. It helps businesses make data-driven decisions about marketing, sales, and customer retention strategies.

## 🧮 1. Simple CLV Formula

- 🎲 Their chance of staying (retention),
- 💰 The value they bring (margin),
- ⏳ The decreasing value of future money (discounting).

Assuming constant margin ( m ), retention rate ( r ), and discount rate ( d ):

$$\text{CLV} = \frac{m \cdot r}{1 + d - r}$$

### 📚 CLV Parameters Table

| Symbol | Name | Description |
|---|---|---|
| $m$ | Margin per period | **Profit** earned **per customer per period**, assumed **constant** over time. |
| $r$ | Retention rate | **Probability** that a customer **remains active** in the next period (value between 0 and 1). |
| $d$ | Discount rate | Reflects how much **time value of money** — future profits are discounted; higher ( d ) means future money is worth less. |

If:

- $r$ is high (close to 1) → Customers stay longer → Higher CLV.
- $m$ is high → More profit per period → Higher CLV.
- $d$ is low → Future money is almost as good as today → Higher CLV.

---

## 📐 Logic Behind the CLV Formula

It is based on geometric series logic. Assume:

- Time is divided into discrete periods (e.g., months, years).
- In each period:
  - The customer stays with probability ( r ).
  - If they stay, they bring margin ( m ).
  - Future profits are discounted at rate ( d ).

So the expected profit stream (CLV) is:

$$\text{CLV} = m \cdot r \cdot \frac{1}{1+d} + m \cdot r^2 \cdot \frac{1}{(1+d)^2} + m \cdot r^3 \cdot \frac{1}{(1+d)^3} + \cdots$$

This is an infinite geometric series of the form:

$$\text{CLV} = m \cdot \sum_{t=1}^{\infty} \left( \frac{r}{1+d} \right)^t$$

This is a geometric series of the form:

$$\sum_{t=1}^{\infty} x^t = \frac{x}{1-x}, \quad \text{for } |x| < 1$$

Let:

$$x = \frac{r}{1+d}$$

Then:

$$\sum_{t=1}^{\infty} \left( \frac{r}{1+d} \right)^t = \frac{\frac{r}{1+d}}{1 - \frac{r}{1+d}} = \frac{r}{1+d-r}$$

So this series converges when:

$$\frac{r}{1+d} < 1$$

Substitute back into the original expression:

$$\text{CLV} = \frac{m \cdot r}{1+d-r}$$

## 📦 2. Discounted Revenue CLV

If we track revenue/profit over time:

$$\text{CLV} = \sum_{t=1}^{T} \frac{r_t \cdot m_t}{(1+d)^t}$$


## 📦 2. Discounted Revenue CLV

If we track revenue/profit over time:

$$\text{CLV} = \sum_{t=1}^{T} \frac{r_t \cdot m_t}{(1+d)^t}$$


- ( $T$ ): customer lifetime (in time periods)
- ( $r_t$ ): retention indicator or probability at time ( t )
- ( $m_t$ ): profit or revenue at time ( t )
- ( $d$ ): discount rate

### How to Estimate Retention and Discount Rate from Raw Data:

- Method 1: Cohort Retention Group customers by when they first transacted (e.g., month joined).

Track whether they made purchases in each future period.

Calculate:

$$r = \frac{\text{Number of customers who stayed}}{\text{Number of customers at start}}$$

- Method 2: Overall Retention Rate If you only have aggregated customer history:

$$r = 1 - ChurnRate$$

And churn rate is:

$$ChurnRate = \frac{\text{Number of lost customers}}{\text{Total customers at the start}}$$

✅ Common Approaches

- Use a fixed discount rate (e.g., 10% annual = d = 0.10) — commonly used if no financial modeling is available.
- Use your company's cost of capital or desired ROI.
- You can also model d based on average risk or inflation in your industry.

Example: If your company expects a 12% return per year, then:

$$d = 0.12$$

If you are calculating monthly:

$$d_{\text{monthly}} = \frac{0.12}{12} = 0.01$$

---

## 📈 3. Transactional CLV (per customer)

Transactional CLV (Customer Lifetime Value) refers to a modeling approach that calculates CLV based on actual, observed transaction data, such as:

- When each customer made purchases
- How much they spent
- How frequently they bought over time

It focuses on tracking customer behavior over time, instead of assuming average behaviors or relying on simplified formulas.

$$\text{CLV}_i = \sum_{j=1}^{n_i} \frac{p_{ij}}{(1+d)^{t_{ij}}}$$

- ( i ): customer index
- ( j ): transaction index
- ( $p_{ij}$ ): profit from transaction ( j ) by customer ( i )
- ( $t_{ij}$ ): time since acquisition or from now
- ( $d$ ): discount rate

| Customer | Transaction | Date | Amount | $D_0$ | $t_{ij}$ (days) |
|---|---|---|---|---|---|
| C123 | $p_{11}$ | 2024-01-01 | $80 | 2024-02-01 | 30 |

| Customer | Transaction | Date | Amount | $D_0$ | $t_{ij}$ (days) |
|---|---|---|---|---|---|
| C123 | $p_{12}$ | 2024-01-17 | $20 | 2024-02-01 | 14 |
| C456 | $p_{21}$ | 2024-01-05 | $200 | 2024-02-01 | 26 |

⌚ For C123:

- $t_{11}$ =12 (1st transaction on day 1)
- $t_{12}$ =12 (2nd transaction on day 17)
- It is used to measure recency: $R_i = T - t_{i,\text{last}}$ →$R_i = 30 - max(t_{ij})$
- It is used to calculate frequency and interpurchase times: $t_{i2} - t_{i1}, \quad t_{i3} - t_{i2}, \quad t_{i4} - t_{i3}, \quad \ldots$
- It is needed in **survival analysis**, where the timing of events (transactions) matters.

| Customer | Total Transactions | Total Amount | Avg Amount |
|---|---|---|---|
| C123 | 2 | 80 + 20 = 100 | $50 |
| C456 | 1 | 200 | $200 |

---

## 💡 4. Predictive CLV (Simplified)

If we estimate future values from behavioral features:

$$\text{CLV}_i = \text{ExpectedFrequency}_i \times \text{ExpectedValue}_i \times \text{ExpectedLifespan}_i$$

Example: If a customer spends $50 on average per purchase, makes 3 purchases per year, and remains a customer for 5 years, then the CLV would be:

$$\begin{gathered}\text{CLV} = \\ \$50 \cdot 3 \cdot 5 = \\ \$750\end{gathered}$$

Each component can be predicted via machine learning or statistics.

---

## 🔮 5. Probabilistic Model (BG/NBD + Gamma-Gamma)

For advanced forecasting, you can use probabilistic models. The most basic formula we use is as follows:

$$\text{CLTV} = \text{Expected Number of Transaction} \cdot \text{Expected Average Profit}$$

### BG/NBD (Buy-Till-You-Die)

Estimates future purchases based on:

- Transaction Process (Buy) → Beta Geometric Distribution
- Dropout process (Till You Die) → Negative Binominal Distribution

Expected future transactions:

$$E[X_i(t)] = \text{Estimated using BG/NBD model}$$

Gamma-Gamma Model (Expected average transaction value):

Estimates average transaction value assuming:

- Transaction amounts follow a Gamma distribution
- Amounts are independent of frequency

$$E[M_i] = \text{Estimated using Gamma-Gamma model}$$

Full model:

$$\text{CLV}_i = E[X_i(t)] \cdot E[M_i]$$

### 📋 CLV Model Summary Table

| Model | Formula | Use Case |
|---|---|---|
| **Simple CLV** | $\frac{m \cdot r}{1+d-r}$ | Subscription, constant margin & churn |
| **Discounted Revenue** | $\sum_{t=1}^{T} \frac{r_t \cdot m_t}{(1+d)^t}$ | Time-based value with discounting |
| **Transactional CLV** | $\sum_{j=1}^{n_i} \frac{p_{ij}}{(1+d)^{t_{ij}}}$ | Historical transaction-level data |
| **Predictive CLV** | $\text{Freq}_i \times \text{Value}_i \times \text{Lifespan}_i$ | ML-based estimation |
| **BG/NBD + Gamma-Gamma** | $E[X_i(t)] \cdot E[M_i]$ | Probabilistic, behavioral prediction |

---

# ✈️ Flight-Based Revenue (FBR) Model

### 📌 What is FBR?

**Flight-Based Revenue (FBR)** is a *quantitative framework* used by countries to estimate and optimize the revenue earned from aircraft that **fly over their airspace** (without landing). It's especially critical for countries located on major international flight routes — like Iran, Turkey, or Egypt.

---

### 🎯 Why FBR Matters

Countries charge airlines **overflight fees** based on factors like:

- Distance flown over the country.
- Aircraft weight or category.
- Route type (e.g., commercial, cargo).

These revenues serve as:

- A **major non-oil income source**.
- A **sensitive indicator** to geopolitical and competitive dynamics.

---

## 🔢 The Core Formula

$$\text{Total Revenue (TR)} = N \times D \times R$$

Where:

- (N): Number of overflights
- (D): Average distance flown over national airspace
- (R): Overflight fee per nautical mile

---

## 🧠 Strategic Components

| Component | Description |
|---|---|
| Overflight Count ((N)) | Depends on route demand, airline routing, and alternatives |
| Distance ((D)) | Typically fixed, based on geography |
| Fee Rate ((R)) | Decision variable—raising it increases revenue but may reduce demand |
| Total Revenue ((TR)) | Output from model—reflects income from overflight charges |

---

## 🧮 Optimization Scenario

If raising the fee from $50 to $55 causes a **3% drop** in flights, but each remaining flight brings in more:

$$\text{New Revenue} = 0.97N \cdot D \cdot 55 = 1.067 \cdot (N \cdot D \cdot 50)$$

This results in a **6.7% increase** in revenue — assuming only a small reduction in traffic volume.

---

## 🧩 Use Cases & Benefits

- 📊 **Revenue Forecasting**
- 🛰️ **Route competition analysis** (e.g., avoiding detours through neighbors)
- ⚖️ **Policy decision-making** (e.g., fee adjustments during sanctions or conflict)

---

## ✅ Conclusion

The **FBR model** is a strategic tool that blends **economics, forecasting, and optimization** to help countries:

- Maximize overflight income
- Compete effectively for international traffic
- Respond intelligently to geopolitical shifts
- Ensure sustainable, data-driven aviation policy

---

## 📊 RFM Analysis

Let:

- ( $C = \{c_1, c_2, \ldots, c_n\}$ ): Set of customers
- ( $T$ ): Reference date (typically the date of the analysis e.g., today's date)
- ( $P_i$ ): Set of purchase dates by customer ( $c_i$ )

---

### 🔁 Recency

Recency value is the number of days a customer takes between two purchases. A smaller value of recency implies that the customer visits the company repeatedly in a short period. Similarly, a greater value implies that the customer is less likely to visit the company shortly.

[ $R_i = T - \max(P_i)$ ]

- Days since the customer's most recent purchase
- Smaller ( $R_i$ ) means more recent activity

---

### 🔂 Frequency

Frequency is defined as the number of purchases a customer makes in a specific period. The higher the value of frequency the more loyal are the customers of the company.

[ $F_i = |P_i|$ ]

- Number of purchases in a defined time window

---

### 💵 Monetary

Monetary is defined as the amount of money spent by the customer during a certain period. The higher the amount of money spent the more revenue they give to the company.

[ $M_i = \sum_{p \in P_i} \text{Amount}_p$ ]

- Total monetary value spent by the customer

---

## 📚 Mathematical Definition of Quantile Binning

Quantile Binning (often shortened as QuantileBin) means dividing a set of numerical values into groups ("bins") based on their quantiles.

- A quantile splits your data into equal-sized parts (e.g., quartiles or quintiles).

Suppose you want to split your data into 4 bins:

- These bins are based on quartiles: 25%, 50%, 75%, 100% points.
- Each bin will cover roughly 25% of the samples.

Similarly:

10 bins → deciles (10% each).

100 bins → percentiles (1% each).

Given a dataset:

$X = \{x_1, x_2, \ldots, x_n\}$

Quantile Binning splits the dataset into ( k ) bins using thresholds:

$q_0, q_1, q_2, \ldots, q_k$

where:

- $q_0 = \min(X)$
- $q_k = \max(X)$
- $q_i$ is the $\frac{i}{k}$ -th quantile of the data.

Each bin is defined as:

$\text{Bin}_i = [q_{i-1}, q_i) \quad \text{for} \quad i = 1, 2, \ldots, k-1$

and

$$\text{Bin}_k = [q_{k-1}, q_k] \quad \text{(including the maximum)}$$

---

✅ In Quantile Binning:

- Each bin contains (approximately) the same number of observations.
- The data points are assigned to bins based on their quantile range.

## 1. Equal Quantiles (Recommended for Uniform Distributions)

**Method:**
Divide the month into equal-sized groups (e.g., quartiles, quintiles).

**Example (Quintiles: 5 Groups):**

| Days | Quantile |
|---|---|
| 1 - 6 | Quantile 1 |

| Days | Quantile |
|---|---|
| 7 - 12 | Quantile 2 |
| 13 - 18 | Quantile 3 |
| 19 - 24 | Quantile 4 |
| 25 - 31 | Quantile 5 |

**Pros:**

- Simple
- Works well if data is evenly distributed

**Cons:**

- Uneven month lengths (e.g., 28 vs. 31 days) can skew groups

## 2. Business-Driven Buckets (Best for Actionable Insights)

**Method:**
Align quantiles with business cycles (e.g., paydays, billing cycles).

**Example:**

| Days | Bucket | Description |
|---|---|---|
| 1 - 10 | Early Month | High spending |
| 11 - 20 | Mid Month | Average activity |
| 21 - 31 | Late Month | Low engagement |

**Pros:**

- Tied to real-world behavior (e.g., users act differently at month-end)

**Cons:**

- Requires domain knowledge

## 3. Percentile-Based (For Skewed Data)

**Method:**
Use percentiles (e.g., 20th/40th/60th/80th) if day behavior isn't uniform.

**Example:**

| Days | Percentile | Description |
|---|---|---|
| 1 - 5 | P20 | Top 20% active days |
| 6 - 12 | P40 | |
| 13 - 20 | P60 | |

| Days | Percentile | Description |
|---|---|---|
| 21 - 31 | P80+ | |

**Pros:**

- Handles outliers (e.g., spikes on specific days)

**Cons:**

- Complex to explain

## 4. Fixed-Day Grouping (For Consistency)

**Method:**
Predefined segments (e.g., weeks).

**Example:**

| Days | Group |
|---|---|
| 1 - 7 | Week 1 |
| 8 - 14 | Week 2 |
| 15 - 21 | Week 3 |
| 22 - 31 | Week 4+ |

**Pros:**

- Easy to compare across months

**Cons:**

- Ignores behavioral nuances

## 🎯 RFM Score (Quantile-Based)

Each value is assigned a score from 1 to 5 (or any other scale), often using quantile binning:

- $R_i^* = \text{QuantileBin}(R_i), \quad$ (lower recency → higher score)
- $F_i^* = \text{QuantileBin}(F_i), \quad$ (higher frequency → higher score)
- $M_i^* = \text{QuantileBin}(M_i)$ (higher monetary → higher score)

Then the RFM score is:

[ $\text{RFM}_i = (R_i^*, F_i^*, M_i^*)$ ]

Or combined into a single number:

[ $\text{RFM}_i = 100 \cdot R_i^* + 10 \cdot F_i^* + M_i^*$ ]

---

📋 RFM Summary Table

| Variable | Formula | Interpretation |
|---|---|---|
| **Recency** | $R_i = T - max(P_i)$ | Days since last purchase |
| **Frequency** | $F_i = \|P_i\|$ | Number of purchases |
| **Monetary** | $M_i = \sum_{p \in P_i} Amount_p$ | Total money spent |

RFM Score Description.

| Score | Characteristics |
|---|---|
| 5 | Potential |
| 4 | Promising |
| 3 | Can't-lose them |
| 2 | At risk |
| 1 | Lost |

# RFM Scoring Example (Quantile-Based)

## Step 1: Sample Data

| Customer | Recency (Days) | Frequency (Orders) | Monetary ($) |
|---|---|---|---|
| A | 30 | 5 | 500 |
| B | 90 | 2 | 200 |
| C | 10 | 8 | 800 |
| D | 120 | 1 | 100 |
| E | 60 | 4 | 400 |

## Step 2: Quantile Scoring (1-5)

### Recency (Lower = Better)

| Customer | R Score |
|---|---|
| C (10) | 5 |
| A (30) | 4 |
| E (60) | 3 |
| B (90) | 2 |
| D (120) | 1 |

### Frequency (Higher = Better)

| Customer | F Score |
|---|---|
| C (8) | 5 |
| A (5) | 4 |
| E (4) | 3 |
| B (2) | 2 |
| D (1) | 1 |

### Monetary (Higher = Better)

| Customer | M Score |
|---|---|
| C (800) | 5 |
| A (500) | 4 |
| E (400) | 3 |
| B (200) | 2 |
| D (100) | 1 |

## Step 3: Combined RFM Scores

| Customer | R | F | M | Total RFM |
|---|---|---|---|---|
| A | 4 | 4 | 4 | 12 |
| B | 2 | 2 | 2 | 6 |
| C | 5 | 5 | 5 | 15 |
| D | 1 | 1 | 1 | 3 |
| E | 3 | 3 | 3 | 9 |

## Step 4: Segmentation

| RFM Range | Segment | Recommended Action |
|---|---|---|
| 13-15 | **Champions** | Reward, retain, upsell |
| 10-12 | **Loyal** | Engage with promotions |
| 7-9 | **Potential** | Re-engage (discounts) |
| 4-6 | **At Risk** | Win-back campaigns |
| 1-3 | **Lost** | Low priority/reactivation |

### Key Insights:

- 🏆 **Top Customer**: C (RFM=15)
- ⚠️ **At Risk**: B (RFM=6)
- ❌ **Lost**: D (RFM=3)

> **Note**: Quantile scoring ensures equal group sizes. Adjust ranges based on your business needs.

---

# My Data Set

In [4]:
```
import numpy as np
import pandas as pd
```

In [5]:
```
df = pd.read_excel('2024-01.xlsx', index_col=0)
```

In [6]:
```
df.head(10)
```

Out[6]:

| airline | entry_time | exit_time | flight_year | flight_month | flight_day | departure | destination | entry_point | entry_coordinate | exit_point | exit_coordinate | reg | wmto | distance | total_charge | discount | final_ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| **UAE** | 2024-01-01 00:00:00 | 2024-01-01 00:39:00 | 2024 | 1 | 1 | Dubai Intl, UAE | Shanghai/ Pudong, China | IMLOT | 25 17 13 N-57 08 00 E | ASVIB | 26 57 24 N-63 18 12 E | A6EEF | 569.000 | 347.34 | 988.67 | 0.00 | |
| **UAE** | 2024-01-01 00:00:00 | 2024-01-01 01:46:00 | 2024 | 1 | 1 | Frankfurt, Main, Germany | Dubai Intl, UAE | BONAM | 38 03 00 N-44 18 00 E | ORSAR | 26 04 30 N-53 57 30 E | A6EUC | 510.000 | 869.69 | 2355.76 | 0.00 | 2 |
| **KMF** | 2024-01-01 00:00:00 | 2024-01-01 00:53:00 | 2024 | 1 | 1 | Jeddah Intl, Saudi Arabia | Kabul Ad, Afghanistan | DASUT | 26 18 32 N-53 11 08 E | RANRU | 30 01 15 N-61 00 48 E | YAKMU | 276.500 | 470.28 | 845.52 | 0.00 | |
| **IGO** | 2024-01-01 00:01:00 | 2024-01-01 00:49:00 | 2024 | 1 | 1 | Abu Dhabi, UAE | Lucknow, India | GABKO | 26 04 04 N-55 47 55 E | ASVIB | 26 57 24 N-63 18 12 E | VTIJC | 79.000 | 406.65 | 241.55 | 0.00 | |
| **ETD** | 2024-01-01 00:01:00 | 2024-01-01 01:46:00 | 2024 | 1 | 1 | Abu Dhabi, UAE | Zurich, Switzerland | RAGAS | 26 35 37 N-52 13 37 E | TESVA | 38 17 09 N-44 29 47 E | A6BMI | 250.836 | 803.12 | 1334.76 | 667.38 | |
| **UAE** | 2024-01-01 00:03:00 | 2024-01-01 00:59:00 | 2024 | 1 | 1 | Kansai Intl, Osaka, JAPAN | Dubai Intl, UAE | DERBO | 29 25 42 N-61 17 01 E | PATAT | 26 16 13 N-56 00 59 E | A6EPW | 351.534 | 337.77 | 741.52 | 0.00 | |
| **UAE** | 2024-01-01 00:04:00 | 2024-01-01 02:14:00 | 2024 | 1 | 1 | Dubai Intl, UAE | London Heathrow, United Kingdom | GABKO | 26 04 04 N-55 47 55 E | DASIS | 38 54 35 N-44 12 30 E | A6EUL | 575.000 | 967.20 | 2766.59 | 0.00 | 2 |
| **UAE** | 2024-01-01 00:04:00 | 2024-01-01 00:51:00 | 2024 | 1 | 1 | Narita Intl, Japan | Dubai Intl, UAE | DERBO | 29 25 42 N-61 17 01 E | PATAT | 26 16 13 N-56 00 59 E | A6EUJ | 575.000 | 337.77 | 966.16 | 0.00 | |
| **UAE** | 2024-01-01 00:04:00 | 2024-01-01 01:36:00 | 2024 | 1 | 1 | Sankt, Peterbuurg, Russian Federation | Dubai Intl, UAE | BATEV | 38 10 05 N-50 14 19 E | ORSAR | 26 04 30 N-53 57 30 E | A6ECK | 351.534 | 750.10 | 1646.72 | 0.00 | 1 |
| **JZR** | 2024-01-01 00:06:00 | 2024-01-01 01:44:00 | 2024 | 1 | 1 | Kathmandu, Nepal | Kuwait, Kuwait | DERBO | 29 25 42 N-61 17 01 E | NANPI | 29 04 57 N-49 31 57 E | 9KCBG | 79.000 | 615.64 | 365.70 | 0.00 | |

In [7]:
```
%load_ext sql
```

In [9]:
```
%sql postgresql://postgres:1234@localhost:5432/lab
%config SqlMagic.displaylimit = None
```

Connecting to 'postgresql://postgres:***@localhost:5432/lab'
displaylimit: Value None will be treated as 0 (no limit)

## FBR

In [10]:
```
%%sql
result <<
select count(*) N, avg(distance) D,(sum(total_charge) / avg(distance)) R
from rfm.f202501 where airline = 'QTR';
```

Running query in 'postgresql://postgres:***@localhost:5432/lab'

1 rows affected.

In [11]:
```
qtr_flight = result.DataFrame()
```

In [12]:
```
qtr_flight
```

Out[12]:

| | n | d | r |
|---|---|---|---|
| **0** | 1189 | 643.462044 | 1594.456658 |

In [13]:
```
qtr_iac_revenue = qtr_flight['n'] * qtr_flight['d'] * qtr_flight['r']
```

In [14]:
```
qtr_iac_revenue.apply(lambda x: f"{x:,.2f}")
```

Out[14]:
```
0    1,219,881,112.26
dtype: object
```

## RFM

In [15]:
```
%%sql
result <<
SELECT
  airline,
  COUNT(*)
FROM
  rfm.f202401
WHERE
  length (airline) = 3
GROUP BY
  airline
ORDER BY
  COUNT desc;
```

Running query in 'postgresql://postgres:***@localhost:5432/lab'

150 rows affected.

In [16]:
```
%%sql
result <<
SELECT
  *
FROM
  (
    SELECT
      airline,
      flight_day,
      COUNT(*) AS flight_count,
      (
        SELECT
          MAX(entry_time)
        FROM
          rfm.f202501
```

```sql
        WHERE
          airline = f202401.airline
      ) last_purchase,
      SUM(total_charge) AS total_charge,
      SUM(final_charge) AS final_charge,
      RANK() OVER (
        PARTITION BY
          flight_day
        ORDER BY
          COUNT(*) DESC
      ) AS RANK
    FROM
      rfm.f202401
    WHERE
      EXISTS (
        SELECT
          1
        FROM
          rfm.f202501
        WHERE
          f202501.airline = f202401.airline
          AND f202501.flight_day = f202401.flight_day

      )
      AND EXISTS (
        SELECT
            1
        FROM
            rfm.f202301
        WHERE
            f202301.airline = f202401.airline
            AND f202301.flight_day = f202401.flight_day
          )
      AND LENGTH (airline) = 3
    GROUP BY
      airline,
      flight_day
  ) sub
WHERE
  RANK <= 5
ORDER BY
  flight_day,
  RANK;
```

Running query in 'postgresql://postgres:***@localhost:5432/lab'

157 rows affected.

In [17]:

```
result
```

Out[17]:

| airline | flight_day | flight_count | last_purchase | total_charge | final_charge | rank |
|---|---|---|---|---|---|---|
| UAE | 1 | 147 | 2025-01-31 23:58:00 | 269997.9200000001 | 269997.9200000001 | 1 |
| FDB | 1 | 126 | 2025-01-31 23:20:00 | 55297.47 | 55297.47 | 2 |
| ETD | 1 | 60 | 2025-01-31 23:55:00 | 59412.97000000001 | 31356.120000000006 | 3 |
| ABY | 1 | 57 | 2025-01-31 23:41:00 | 21174.74 | 21174.74 | 4 |
| QTR | 1 | 51 | 2025-01-31 23:54:00 | 43247.020000000004 | 43247.020000000004 | 5 |
| FDB | 2 | 134 | 2025-01-31 23:20:00 | 60582.39000000003 | 60582.39000000003 | 1 |
| UAE | 2 | 112 | 2025-01-31 23:58:00 | 179593.67000000004 | 179593.67000000004 | 2 |
| ABY | 2 | 48 | 2025-01-31 23:41:00 | 17438.319999999992 | 17438.319999999992 | 3 |
| THY | 2 | 46 | 2025-01-31 23:45:00 | 77646.97 | 77646.97 | 4 |
| QTR | 2 | 44 | 2025-01-31 23:54:00 | 41590.94999999998 | 41590.94999999998 | 5 |
| UAE | 3 | 135 | 2025-01-31 23:58:00 | 214169.32000000012 | 214169.32000000012 | 1 |
| FDB | 3 | 133 | 2025-01-31 23:20:00 | 60766.609999999986 | 60766.609999999986 | 2 |
| ABY | 3 | 54 | 2025-01-31 23:41:00 | 20016.07 | 20016.07 | 3 |
| QTR | 3 | 46 | 2025-01-31 23:54:00 | 43931.450000000004 | 43931.450000000004 | 4 |
| ETD | 3 | 46 | 2025-01-31 23:55:00 | 41408.56 | 22315.030000000002 | 4 |
| UAE | 4 | 116 | 2025-01-31 23:58:00 | 190902.53000000012 | 190902.53000000012 | 1 |
| FDB | 4 | 114 | 2025-01-31 23:20:00 | 50617.669999999984 | 50617.669999999984 | 2 |
| ABY | 4 | 52 | 2025-01-31 23:41:00 | 21237.559999999994 | 21237.559999999994 | 3 |
| QTR | 4 | 44 | 2025-01-31 23:54:00 | 39269.66 | 39269.66 | 4 |
| THY | 4 | 43 | 2025-01-31 23:45:00 | 75196.20000000003 | 75196.20000000003 | 5 |
| UAE | 5 | 133 | 2025-01-31 23:58:00 | 213752.71000000008 | 213752.71000000008 | 1 |
| FDB | 5 | 127 | 2025-01-31 23:20:00 | 58568.01 | 58568.01 | 2 |
| QTR | 5 | 58 | 2025-01-31 23:54:00 | 52481.33 | 52481.33 | 3 |
| ABY | 5 | 55 | 2025-01-31 23:41:00 | 21586.649999999994 | 21586.649999999994 | 4 |
| THY | 5 | 40 | 2025-01-31 23:45:00 | 71825.25 | 71825.25 | 5 |
| FDB | 6 | 138 | 2025-01-31 23:20:00 | 60531.34000000001 | 60531.34000000001 | 1 |
| UAE | 6 | 134 | 2025-01-31 23:58:00 | 232153.40000000014 | 232153.40000000014 | 2 |
| ABY | 6 | 62 | 2025-01-31 23:41:00 | 25365.659999999996 | 25365.659999999996 | 3 |
| THY | 6 | 49 | 2025-01-31 23:45:00 | 84196.01 | 84196.01 | 4 |
| QTR | 6 | 44 | 2025-01-31 23:54:00 | 36020.42 | 36020.42 | 5 |
| ETD | 6 | 44 | 2025-01-31 23:55:00 | 47015.870000000024 | 24174.53000000001 | 5 |
| FDB | 7 | 131 | 2025-01-31 23:20:00 | 60605.100000000035 | 60605.100000000035 | 1 |

| airline | flight_day | flight_count | last_purchase | total_charge | final_charge | rank |
|---|---|---|---|---|---|---|
| UAE | 7 | 128 | 2025-01-31 23:58:00 | 221609.92000000007 | 221609.92000000007 | 2 |
| ABY | 7 | 62 | 2025-01-31 23:41:00 | 23557.489999999987 | 23557.489999999987 | 3 |
| ETD | 7 | 56 | 2025-01-31 23:55:00 | 56131.51 | 29710.219999999998 | 4 |
| THY | 7 | 47 | 2025-01-31 23:45:00 | 82764.67000000001 | 82764.67000000001 | 5 |
| UAE | 8 | 131 | 2025-01-31 23:58:00 | 204750.26000000004 | 204750.26000000004 | 1 |
| FDB | 8 | 125 | 2025-01-31 23:20:00 | 55020.4 | 55020.4 | 2 |
| ABY | 8 | 57 | 2025-01-31 23:41:00 | 21253.779999999988 | 21253.779999999988 | 3 |
| QTR | 8 | 45 | 2025-01-31 23:54:00 | 32319.38 | 32319.38 | 4 |
| ETD | 8 | 44 | 2025-01-31 23:55:00 | 40641.66 | 21845.61 | 5 |
| FDB | 9 | 126 | 2025-01-31 23:20:00 | 55572.65000000002 | 55572.65000000002 | 1 |
| UAE | 9 | 114 | 2025-01-31 23:58:00 | 195689.60999999996 | 195689.60999999996 | 2 |
| ABY | 9 | 52 | 2025-01-31 23:41:00 | 20708.739999999998 | 20708.739999999998 | 3 |
| QTR | 9 | 51 | 2025-01-31 23:54:00 | 41045.049999999996 | 41045.049999999996 | 4 |
| THY | 9 | 44 | 2025-01-31 23:45:00 | 73131.73 | 73131.73 | 5 |
| FDB | 10 | 119 | 2025-01-31 23:20:00 | 53000.28000000003 | 53000.28000000003 | 1 |
| UAE | 10 | 113 | 2025-01-31 23:58:00 | 178711.79000000015 | 178711.79000000015 | 2 |
| QTR | 10 | 56 | 2025-01-31 23:54:00 | 43905.189999999995 | 43905.189999999995 | 3 |
| ABY | 10 | 54 | 2025-01-31 23:41:00 | 19687.13000000001 | 19687.13000000001 | 4 |
| THY | 10 | 48 | 2025-01-31 23:45:00 | 81161.3 | 81161.3 | 5 |
| FDB | 11 | 116 | 2025-01-31 23:20:00 | 47651.499999999985 | 47651.499999999985 | 1 |
| UAE | 11 | 100 | 2025-01-31 23:58:00 | 164440.83000000005 | 164440.83000000005 | 2 |
| ETD | 11 | 48 | 2025-01-31 23:55:00 | 47612.77000000001 | 25440.060000000005 | 3 |
| ABY | 11 | 47 | 2025-01-31 23:41:00 | 17233.669999999995 | 17233.669999999995 | 4 |
| THY | 11 | 47 | 2025-01-31 23:45:00 | 83032.74999999999 | 83032.74999999999 | 4 |
| UAE | 12 | 146 | 2025-01-31 23:58:00 | 265449.8200000001 | 265449.8200000001 | 1 |
| FDB | 12 | 124 | 2025-01-31 23:20:00 | 55285.26 | 55285.26 | 2 |
| ABY | 12 | 57 | 2025-01-31 23:41:00 | 22052.31 | 22052.31 | 3 |
| ETD | 12 | 56 | 2025-01-31 23:55:00 | 57806.24000000003 | 30276.31000000001 | 4 |
| QTR | 12 | 53 | 2025-01-31 23:54:00 | 48456.51000000001 | 48456.51000000001 | 5 |
| FDB | 13 | 131 | 2025-01-31 23:20:00 | 57348.09 | 57348.09 | 1 |
| UAE | 13 | 118 | 2025-01-31 23:58:00 | 192805.39000000013 | 192805.39000000013 | 2 |
| QTR | 13 | 59 | 2025-01-31 23:54:00 | 51573.56999999999 | 51573.56999999999 | 3 |

| airline | flight_day | flight_count | last_purchase | total_charge | final_charge | rank |
|---|---|---|---|---|---|---|
| ABY | 13 | 55 | 2025-01-31 23:41:00 | 21137.21 | 21137.21 | 4 |
| THY | 13 | 42 | 2025-01-31 23:45:00 | 68724.98999999999 | 68724.98999999999 | 5 |
| UAE | 14 | 115 | 2025-01-31 23:58:00 | 190780.87000000017 | 190780.87000000017 | 1 |
| FDB | 14 | 111 | 2025-01-31 23:20:00 | 48786.67999999999 | 48786.67999999999 | 2 |
| ABY | 14 | 59 | 2025-01-31 23:41:00 | 23138.25999999999 | 23138.25999999999 | 3 |
| QTR | 14 | 53 | 2025-01-31 23:54:00 | 43529.21 | 43529.21 | 4 |
| THY | 14 | 46 | 2025-01-31 23:45:00 | 79019.53000000001 | 79019.53000000001 | 5 |
| UAE | 15 | 111 | 2025-01-31 23:58:00 | 180461.25000000012 | 180461.25000000012 | 1 |
| FDB | 15 | 111 | 2025-01-31 23:20:00 | 48260.40000000001 | 48260.40000000001 | 1 |
| QTR | 15 | 53 | 2025-01-31 23:54:00 | 48646.43999999999 | 48646.43999999999 | 3 |
| ABY | 15 | 53 | 2025-01-31 23:41:00 | 18834.609999999986 | 18834.609999999986 | 3 |
| ETD | 15 | 49 | 2025-01-31 23:55:00 | 55587.59000000002 | 29871.040000000008 | 5 |
| UAE | 16 | 126 | 2025-01-31 23:58:00 | 194575.01000000013 | 194575.01000000013 | 1 |
| FDB | 16 | 122 | 2025-01-31 23:20:00 | 53655.20000000001 | 53655.20000000001 | 2 |
| QTR | 16 | 64 | 2025-01-31 23:54:00 | 62611.05000000001 | 62611.05000000001 | 3 |
| ETD | 16 | 55 | 2025-01-31 23:55:00 | 58365.930000000015 | 30456.090000000007 | 4 |
| ABY | 16 | 52 | 2025-01-31 23:41:00 | 18948.67 | 18948.67 | 5 |
| UAE | 17 | 136 | 2025-01-31 23:58:00 | 194732.75000000015 | 194732.75000000015 | 1 |
| FDB | 17 | 115 | 2025-01-31 23:20:00 | 50918.28000000002 | 50918.28000000002 | 2 |
| QTR | 17 | 65 | 2025-01-31 23:54:00 | 60208.149999999994 | 60208.149999999994 | 3 |
| ETD | 17 | 56 | 2025-01-31 23:55:00 | 59658.31000000002 | 31091.900000000005 | 4 |
| ABY | 17 | 48 | 2025-01-31 23:41:00 | 17779.559999999994 | 17779.559999999994 | 5 |
| FDB | 18 | 118 | 2025-01-31 23:20:00 | 52125.38999999999 | 52125.38999999999 | 1 |
| UAE | 18 | 110 | 2025-01-31 23:58:00 | 182656.74000000008 | 182656.74000000008 | 2 |
| QTR | 18 | 52 | 2025-01-31 23:54:00 | 44477.909999999996 | 44477.909999999996 | 3 |
| THY | 18 | 49 | 2025-01-31 23:45:00 | 80643.0 | 80643.0 | 4 |
| ETD | 18 | 45 | 2025-01-31 23:55:00 | 45712.8 | 23961.02 | 5 |
| FDB | 19 | 118 | 2025-01-31 23:20:00 | 51849.22000000001 | 51849.22000000001 | 1 |
| UAE | 19 | 88 | 2025-01-31 23:58:00 | 178794.39 | 178794.39 | 2 |
| QTR | 19 | 65 | 2025-01-31 23:54:00 | 56612.09999999998 | 56612.09999999998 | 3 |
| ETD | 19 | 55 | 2025-01-31 23:55:00 | 55635.12000000001 | 28819.79 | 4 |
| ABY | 19 | 55 | 2025-01-31 23:41:00 | 20129.73 | 20129.73 | 4 |

| airline | flight_day | flight_count | last_purchase | total_charge | final_charge | rank |
|---|---|---|---|---|---|---|
| FDB | 20 | 122 | 2025-01-31 23:20:00 | 51814.78000000005 | 51814.78000000005 | 1 |
| UAE | 20 | 105 | 2025-01-31 23:58:00 | 198704.9800000001 | 198704.9800000001 | 2 |
| ETD | 20 | 52 | 2025-01-31 23:55:00 | 54995.49000000001 | 28006.450000000004 | 3 |
| QTR | 20 | 51 | 2025-01-31 23:54:00 | 46215.09 | 46215.09 | 4 |
| ABY | 20 | 50 | 2025-01-31 23:41:00 | 18364.549999999996 | 18364.549999999996 | 5 |
| UAE | 21 | 115 | 2025-01-31 23:58:00 | 186663.06 | 186663.06 | 1 |
| FDB | 21 | 114 | 2025-01-31 23:20:00 | 50292.53 | 50292.53 | 2 |
| ABY | 21 | 60 | 2025-01-31 23:41:00 | 22917.5 | 22917.5 | 3 |
| QTR | 21 | 57 | 2025-01-31 23:54:00 | 44527.79000000001 | 44527.79000000001 | 4 |
| ETD | 21 | 54 | 2025-01-31 23:55:00 | 57386.200000000026 | 29855.19000000001 | 5 |
| UAE | 22 | 119 | 2025-01-31 23:58:00 | 188702.75000000012 | 188702.75000000012 | 1 |
| FDB | 22 | 113 | 2025-01-31 23:20:00 | 49353.01000000001 | 49353.01000000001 | 2 |
| ETD | 22 | 56 | 2025-01-31 23:55:00 | 59789.200000000026 | 31325.370000000014 | 3 |
| QTR | 22 | 50 | 2025-01-31 23:54:00 | 43907.31 | 43907.31 | 4 |
| ABY | 22 | 46 | 2025-01-31 23:41:00 | 16440.800000000003 | 16440.800000000003 | 5 |
| FDB | 23 | 124 | 2025-01-31 23:20:00 | 53135.800000000025 | 53135.800000000025 | 1 |
| UAE | 23 | 111 | 2025-01-31 23:58:00 | 178835.84000000008 | 178835.84000000008 | 2 |
| ETD | 23 | 54 | 2025-01-31 23:55:00 | 54578.19000000002 | 28562.030000000006 | 3 |
| QTR | 23 | 53 | 2025-01-31 23:54:00 | 41736.459999999985 | 41736.459999999985 | 4 |
| THY | 23 | 49 | 2025-01-31 23:45:00 | 79302.07999999999 | 79302.07999999999 | 5 |
| UAE | 24 | 124 | 2025-01-31 23:58:00 | 204449.57000000007 | 204449.57000000007 | 1 |
| FDB | 24 | 114 | 2025-01-31 23:20:00 | 51453.99000000001 | 51453.99000000001 | 2 |
| ETD | 24 | 62 | 2025-01-31 23:55:00 | 64934.59000000002 | 33775.47000000001 | 3 |
| QTR | 24 | 50 | 2025-01-31 23:54:00 | 45559.819999999985 | 45559.819999999985 | 4 |
| THY | 24 | 45 | 2025-01-31 23:45:00 | 74877.05999999998 | 74877.05999999998 | 5 |
| ABY | 24 | 45 | 2025-01-31 23:41:00 | 16443.899999999998 | 16443.899999999998 | 5 |
| FDB | 25 | 115 | 2025-01-31 23:20:00 | 48796.100000000006 | 48796.100000000006 | 1 |
| UAE | 25 | 108 | 2025-01-31 23:58:00 | 186655.82000000007 | 186655.82000000007 | 2 |
| QTR | 25 | 49 | 2025-01-31 23:54:00 | 41923.00999999998 | 41923.00999999998 | 3 |
| ABY | 25 | 46 | 2025-01-31 23:41:00 | 16328.409999999994 | 16328.409999999994 | 4 |
| ETD | 25 | 46 | 2025-01-31 23:55:00 | 49100.13000000001 | 27094.53 | 4 |
| UAE | 26 | 119 | 2025-01-31 23:58:00 | 192726.78000000014 | 192726.78000000014 | 1 |

| airline | flight_day | flight_count | last_purchase | total_charge | final_charge | rank |
|---|---|---|---|---|---|---|
| FDB | 26 | 115 | 2025-01-31 23:20:00 | 51584.69000000002 | 51584.69000000002 | 2 |
| QTR | 26 | 60 | 2025-01-31 23:54:00 | 59109.27 | 59109.27 | 3 |
| ABY | 26 | 52 | 2025-01-31 23:41:00 | 18999.219999999998 | 18999.219999999998 | 4 |
| ETD | 26 | 50 | 2025-01-31 23:55:00 | 55174.83000000003 | 28385.52000000001 | 5 |
| FDB | 27 | 124 | 2025-01-31 23:20:00 | 54187.32999999998 | 54187.32999999998 | 1 |
| UAE | 27 | 99 | 2025-01-31 23:58:00 | 147916.34999999998 | 147916.34999999998 | 2 |
| ABY | 27 | 50 | 2025-01-31 23:41:00 | 18584.849999999995 | 18584.849999999995 | 3 |
| THY | 27 | 50 | 2025-01-31 23:45:00 | 77540.10999999999 | 77540.10999999999 | 3 |
| ETD | 27 | 49 | 2025-01-31 23:55:00 | 47147.88999999999 | 25202.48999999999 | 5 |
| FDB | 28 | 110 | 2025-01-31 23:20:00 | 48885.570000000014 | 48885.570000000014 | 1 |
| UAE | 28 | 102 | 2025-01-31 23:58:00 | 170163.65000000005 | 170163.65000000005 | 2 |
| ABY | 28 | 54 | 2025-01-31 23:41:00 | 19617.199999999997 | 19617.199999999997 | 3 |
| ETD | 28 | 53 | 2025-01-31 23:55:00 | 58245.85000000001 | 30207.820000000003 | 4 |
| THY | 28 | 49 | 2025-01-31 23:45:00 | 83824.98 | 83824.98 | 5 |
| FDB | 29 | 117 | 2025-01-31 23:20:00 | 50662.72000000001 | 50662.72000000001 | 1 |
| UAE | 29 | 100 | 2025-01-31 23:58:00 | 160181.15999999997 | 160181.15999999997 | 2 |
| QTR | 29 | 44 | 2025-01-31 23:54:00 | 38589.19999999999 | 38589.19999999999 | 3 |
| ABY | 29 | 44 | 2025-01-31 23:41:00 | 15572.289999999999 | 15572.289999999999 | 3 |
| ETD | 29 | 39 | 2025-01-31 23:55:00 | 42242.22 | 22041.360000000008 | 5 |
| FDB | 30 | 115 | 2025-01-31 23:20:00 | 51625.29000000001 | 51625.29000000001 | 1 |
| UAE | 30 | 97 | 2025-01-31 23:58:00 | 160642.26999999996 | 160642.26999999996 | 2 |
| ETD | 30 | 48 | 2025-01-31 23:55:00 | 51325.160000000025 | 26528.96000000001 | 3 |
| ABY | 30 | 47 | 2025-01-31 23:41:00 | 16934.429999999997 | 16934.429999999997 | 4 |
| THY | 30 | 47 | 2025-01-31 23:45:00 | 75854.77000000002 | 75854.77000000002 | 4 |
| FDB | 31 | 113 | 2025-01-31 23:20:00 | 50745.920000000035 | 50745.920000000035 | 1 |
| UAE | 31 | 107 | 2025-01-31 23:58:00 | 165480.85000000006 | 165480.85000000006 | 2 |
| ETD | 31 | 50 | 2025-01-31 23:55:00 | 52091.150000000016 | 27683.520000000008 | 3 |
| ABY | 31 | 45 | 2025-01-31 23:41:00 | 16539.409999999996 | 16539.409999999996 | 4 |
| THY | 31 | 38 | 2025-01-31 23:45:00 | 67269.31999999998 | 67269.31999999998 | 5 |

In [18]:
```
df_flight = result.DataFrame()
```

In [19]:
```
df_flight
```

Out[19]:

| | airline | flight_day | flight_count | last_purchase | total_charge | final_charge | rank |
|---|---|---|---|---|---|---|---|
| **0** | UAE | 1 | 147 | 2025-01-31 23:58:00 | 269997.92 | 269997.92 | 1 |
| **1** | FDB | 1 | 126 | 2025-01-31 23:20:00 | 55297.47 | 55297.47 | 2 |
| **2** | ETD | 1 | 60 | 2025-01-31 23:55:00 | 59412.97 | 31356.12 | 3 |
| **3** | ABY | 1 | 57 | 2025-01-31 23:41:00 | 21174.74 | 21174.74 | 4 |
| **4** | QTR | 1 | 51 | 2025-01-31 23:54:00 | 43247.02 | 43247.02 | 5 |
| **...** | ... | ... | ... | ... | ... | ... | ... |
| **152** | FDB | 31 | 113 | 2025-01-31 23:20:00 | 50745.92 | 50745.92 | 1 |
| **153** | UAE | 31 | 107 | 2025-01-31 23:58:00 | 165480.85 | 165480.85 | 2 |
| **154** | ETD | 31 | 50 | 2025-01-31 23:55:00 | 52091.15 | 27683.52 | 3 |
| **155** | ABY | 31 | 45 | 2025-01-31 23:41:00 | 16539.41 | 16539.41 | 4 |
| **156** | THY | 31 | 38 | 2025-01-31 23:45:00 | 67269.32 | 67269.32 | 5 |

157 rows × 7 columns

In [20]:

```python
# Reference date: the max(last_purchase) + 1 day
reference_date = df_flight['last_purchase'].max() + pd.Timedelta(days=1)

# Build the RFM metrics
rfm = df_flight.groupby('airline').agg({
    'last_purchase': lambda x: (reference_date - x.max()).days,  # Recency in days
    'flight_count': 'sum',                                       # Total number of flights
    'final_charge': 'sum'                                        # Total revenue
})

rfm.rename(columns={
    'last_purchase': 'Recency',
    'flight_count': 'Frequency',
    'final_charge': 'Monetary'
}, inplace=True)

# Score each part
rfm['R_Score'] = pd.qcut(rfm['Recency'].rank(method='first'), 5, labels=[5,4,3,2,1])
rfm['F_Score'] = pd.qcut(rfm['Frequency'].rank(method='first'), 5, labels=[1,2,3,4,5])
rfm['M_Score'] = pd.qcut(rfm['Monetary'].rank(method='first'), 5, labels=[1,2,3,4,5])

# Adjust Recency by inverting the score
rfm['R_Score'] = 6 - rfm['R_Score'].astype(int)  # If R_Score is from 1 (best) to 5 (worst)

# Combine into RFM Segment
rfm['RFM_Segment'] = rfm['R_Score'].astype(str) + rfm['F_Score'].astype(str) + rfm['M_Score'].astype(str)
rfm['RFM_Score'] = rfm[['R_Score', 'F_Score', 'M_Score']].sum(axis=1)

rfm = rfm.sort_values('RFM_Score', ascending=False)

rfm.head()
```

Out[20]:

| airline | Recency | Frequency | Monetary | R_Score | F_Score | M_Score | RFM_Segment | RFM_Score |
|---|---|---|---|---|---|---|---|---|
| UAE | 1 | 3619 | 5987151.26 | 5 | 4 | 5 | 545 | 14 |
| FDB | 1 | 3735 | 1648979.67 | 2 | 5 | 4 | 254 | 11 |
| THY | 1 | 779 | 1316010.72 | 4 | 1 | 3 | 413 | 8 |
| QTR | 1 | 1317 | 1151493.34 | 3 | 2 | 2 | 322 | 7 |
| ABY | 1 | 1518 | 568022.72 | 1 | 3 | 1 | 131 | 5 |

In [21]:

```
# Mapping based on image
score_description = {
    5: 'Potential',
    4: 'Promising',
    3: "Can’t-lose them",
    2: 'At risk',
    1: 'Lost'
}
# Ensure scores are integers
rfm[['R_Score', 'F_Score', 'M_Score']] = rfm[['R_Score', 'F_Score', 'M_Score']].astype(int)
# Map descriptions
rfm['R_Label'] = rfm['R_Score'].map(score_description)
rfm['F_Label'] = rfm['F_Score'].map(score_description)
rfm['M_Label'] = rfm['M_Score'].map(score_description)


# Sort by segment importance if you want
rfm = rfm.sort_values(['RFM_Score'], ascending=[False])

rfm.head(15)
```

Out[21]:

| airline | Recency | Frequency | Monetary | R_Score | F_Score | M_Score | RFM_Segment | RFM_Score | R_Label | F_Label | M_Label |
|---|---|---|---|---|---|---|---|---|---|---|---|
| UAE | 1 | 3619 | 5987151.26 | 5 | 4 | 5 | 545 | 14 | Potential | Promising | Potential |
| FDB | 1 | 3735 | 1648979.67 | 2 | 5 | 4 | 254 | 11 | At risk | Potential | Promising |
| THY | 1 | 779 | 1316010.72 | 4 | 1 | 3 | 413 | 8 | Promising | Lost | Can’t-lose them |
| QTR | 1 | 1317 | 1151493.34 | 3 | 2 | 2 | 322 | 7 | Can’t-lose them | At risk | At risk |
| ABY | 1 | 1518 | 568022.72 | 1 | 3 | 1 | 131 | 5 | Lost | Can’t-lose them | Lost |
| ETD | 1 | 1227 | 667986.43 | 1 | 1 | 1 | 111 | 3 | Lost | Lost | Lost |

In [22]:

```
import matplotlib.pyplot as plt
import seaborn as sns
```

In [23]:

```
# 1. Filter for top rank = 1
top_airlines = df_flight[df_flight['rank'] == 1]

# 2. Count how many times each airline was rank 1
rank_counts = top_airlines['airline'].value_counts()
```

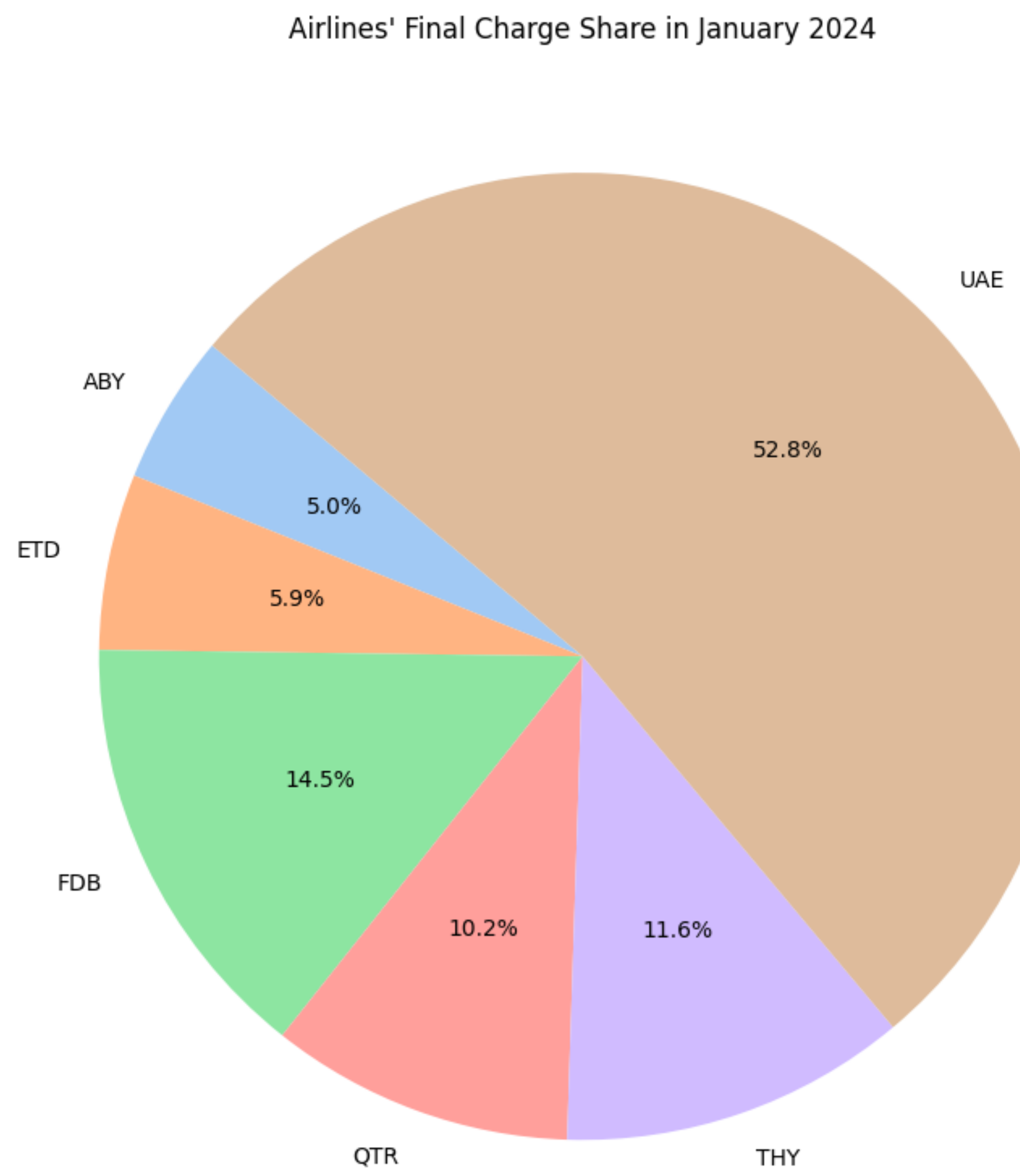
```python
# 3. Sum of final_charge for each airline for whole month
monetary_sum = df_flight.groupby('airline')['final_charge'].sum()

# 4. Combine both into a DataFrame
combined = pd.DataFrame({
    'top_rank_count': rank_counts,
    'total_final_charge': monetary_sum
}).fillna(0)

# Optional: Normalize if needed
combined['total_final_charge'] = combined['total_final_charge'].astype(float)
combined['score'] = combined['top_rank_count'] + (combined['total_final_charge'] / combined['total_final_charge'].max())

# 5. Plot Pie chart for monetary contribution
plt.figure(figsize=(8, 8))
colors = sns.color_palette('pastel')[0:len(combined)]

plt.pie(
    combined['total_final_charge'],
    labels=combined.index,
    autopct='%1.1f%%',
    startangle=140,
    colors=colors
)
plt.title("Airlines' Final Charge Share in January 2024")
plt.tight_layout()
plt.show()
```




---

In [24]:
```
import folium
from geopy.distance import geodesic
```

In [25]:
```
# Helper to convert DMS (degrees, minutes, seconds) to decimal
def dms_to_decimal(d, m, s, direction):
    decimal = d + m / 60 + s / 3600
    if direction in ['S', 'W']:
        return -decimal
    return decimal
```

In [26]:
```
def parse_dms_string(dms_str):
    # Split on the "-" between lat and lon
    lat_str, lon_str = dms_str.split('-')
    # Extract components
    lat_parts = lat_str.strip().split()
    lon_parts = lon_str.strip().split()
    return (
        (int(lat_parts[0]), int(lat_parts[1]), int(lat_parts[2]), lat_parts[3]),
        (int(lon_parts[0]), int(lon_parts[1]), int(lon_parts[2]), lon_parts[3])
    )
```

In [27]:
```
%%sql result <<
select point, coordinate
from rfm.points;
```

Running query in 'postgresql://postgres:***@localhost:5432/lab'

505 rows affected.

In [28]:
```
df2 = result.DataFrame()
```

In [29]:
```
%%sql
result <<
SELECT
  max(distance) max,entry_point,exit_point
FROM
  rfm.f202401
WHERE
  length (airline) = 3
GROUP BY
  entry_point,exit_point
ORDER BY
   max(distance) desc limit 1;
```

Running query in 'postgresql://postgres:***@localhost:5432/lab'

1 rows affected.

In [30]:
```
result
```

Out[30]:

| max | entry_point | exit_point |
|---|---|---|
| 1213.97 | AGINA | ASVIB |

In [31]:
```
%%sql
result <<
select departure,destination
from rfm.f202401
where entry_point = 'AGINA' AND exit_point = 'ASVIB';
```

Running query in 'postgresql://postgres:***@localhost:5432/lab'

1 rows affected.

In [32]:
```
result
```

Out[32]:

| departure | destination |
|---|---|
| Frankfurt, Main, Germany | Mumbai, Jawaharlal Lal Nehru, India |

In [33]:
```
# Coordinates dictionary
waypoints = dict(zip(df2['point'], df2['coordinate']))
```

In [34]:
```
# Convert all to decimal degrees
coords = {}
for name, coord_str in waypoints.items():
    lat_dms, lon_dms = parse_dms_string(coord_str)
    lat = dms_to_decimal(*lat_dms)
    lon = dms_to_decimal(*lon_dms)
    coords[name] = (lat, lon)
```

In [35]:
```
# Function to plot two points with distance
def plot_two_points_with_distance(name1, name2):
    if name1 not in coords or name2 not in coords:
        print("One or both point names not found.")
        return

    lat1, lon1 = coords[name1]
    lat2, lon2 = coords[name2]

    # Calculate distance
    point1 = (lat1, lon1)
    point2 = (lat2, lon2)
    distance = geodesic(point1, point2).nautical  # ✅ Gives you distance in nautical miles

    # Midpoint for map centering
    mid_lat = (lat1 + lat2) / 2
    mid_lon = (lon1 + lon2) / 2

    # Create the map centered at the midpoint
    m = folium.Map(location=[mid_lat, mid_lon], zoom_start=6)

    # Add markers for both points
    folium.Marker([lat1, lon1], popup=name1, tooltip=name1).add_to(m)
    folium.Marker([lat2, lon2], popup=name2, tooltip=name2).add_to(m)

    # Draw a line between the two points
    folium.PolyLine(locations=[[lat1, lon1], [lat2, lon2]], color="blue").add_to(m)

    # Add the distance label on the map near the line
    folium.Marker(
        location=[(lat1 + lat2 + 0.4)/2, (lon1 + lon2 + 0.4)/2],
        icon=folium.DivIcon(html=f'<div style="font-size: 12px; color: black;">{distance:.2f} N.M</div>')
    ).add_to(m)
    return m
```

In [36]:
```
plot_two_points_with_distance('AGINA', 'ASVIB')
```

Out[36]:

403r
Access blocked
Referer is required by tile usage policy of OpenStreetMap's volunteer-run servers: osm.wiki/Blocked
1214.42
N.M
Leaflet | © OpenStreetMap contributors

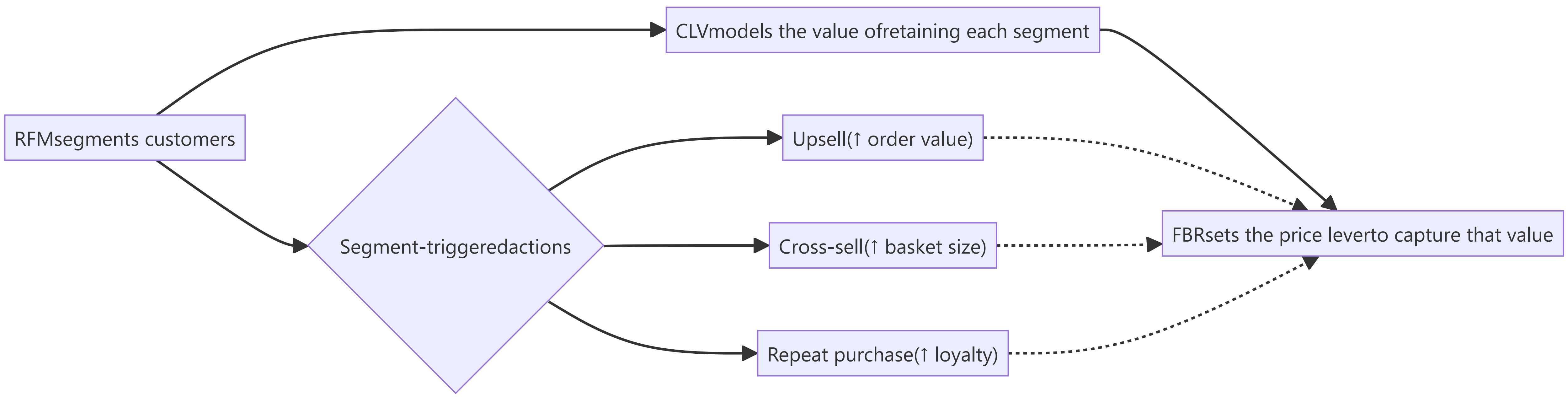

Comparing RFM, CLV, and FBR through chaos lens

- آیا این مدل‌ها می‌توانند رفتار آشوبناک از خود نشان دهند؟
- چه شرایطی برای بروز آشوب در این مدل‌ها لازم است؟
- نقش پارامترهای کنترلی در پایداری یا ناپایداری سیستم چیست؟

- آیا تغییرات کوچک در ورودی‌ها می‌تواند به تغییرات بزرگ در خروجی منجر شود؟
- کدام مدل ذاتاً برای تحلیل آشوب مناسب‌تر است؟
- چه تفاوتی میان مدل‌های تک‌بعدی و چندبعدی از نظر امکان بروز آشوب وجود دارد؟
- برای تحلیل آشوب در این مدل‌ها از چه ابزارهای ریاضی می‌توان استفاده کرد؟
- چگونه می‌توان رفتار آشوبناک را از رفتار تصادفی تشخیص داد؟
- آشوب چه اثری بر پیش‌بینی‌پذیری سیستم دارد؟
- شناخت آشوب چه کمکی به تصمیم‌گیری و کنترل سیستم می‌کند؟
- چه راه‌هایی برای جلوگیری یا کنترل ورود سیستم به ناحیه آشوب وجود دارد؟
- ارزش علمی و کاربردی تحلیل آشوب در این مدل‌ها چیست؟

## Revenue Pipeline: RFM → CLV → FBR

the actual theorems (Period-3 Theorem, Sharkovsky, the orbit diagram) apply to specific quadratic maps, not literally to CRM systems. The value here is using the same iterative structure to see when a revenue policy becomes unpredictable in the technical sense.




```
┌─────────────┐     ┌────────────────┐     ┌─────────────────┐
│     RFM     │ ──► │      CLV       │ ──► │      FBR        │
│  segments   │     │ models value of│     │ sets price lever│
│  customers  │     │ retaining them │     │ to capture value│
└─────────────┘     └────────────────┘     └─────────────────┘
       │                                          ▲
       │                                          │
       │         triggers per-segment actions     │
       ▼                                          │
┌──────────────────────────────────────────┐      │
│  Upsell (↑ order value)                  │──────┤
│  Cross-sell (↑ basket size)              │──────┤
│  Repeat purchase (↑ loyalty)             │──────┘
└──────────────────────────────────────────┘
```

We turn each model into a **feedback map** — this period's output feeds next period's input — the same setup as `Qc(x) = x² + c` or the logistic family `Fλ(x) = λx(1 − x)`.

---

## FBR — the natural chaos candidate

FBR already contains a feedback loop: raising the fee rate `R` reduces the number of flights `N` via elasticity. If `R` reacts each period to *last period's* traffic response, collapsing this into a single-variable recursion gives:

$$N_{n+1} = \lambda \cdot N_n \cdot (1 - N_n / N_{\max})$$

where `λ` packages together the elasticity $E_d$ and your fee-adjustment aggressiveness. This is structurally identical to the logistic family.

From the orbit diagram:

- **λ < 3**: traffic settles to a stable equilibrium (single fixed point) — predictable revenue.

- **3 ≤ λ < 3.57…**: period-doubling bifurcations — traffic oscillates in 2-, 4-, 8-cycles (e.g., alternating high/low booking quarters).
- **λ ≥ 3.57… (Feigenbaum point)**: **chaos onset**. Small changes in fee-adjustment aggressiveness produce qualitatively different long-run traffic/revenue patterns.

**Checking the three criteria in this regime:**

- **Sensitive dependence on Initial Conditions**: two nearly-identical initial fee rates `R₀` diverge into completely different traffic trajectories after enough quarters — the same phenomenon as the orbit-divergence table in Fig. 10.7. Practically: your 8-quarter-ahead FBR forecast is worthless once `λ` crosses the threshold.
- **Transitivity**: the traffic level can be driven, by some sequence of fee decisions, arbitrarily close to *any* target level — there's no "safe zone" it's permanently locked out of.
- **Dense periodic points**: for any traffic level you want to hit, there's a periodic fee-schedule (a seasonal pricing cycle) landing arbitrarily close to it — but these cycles are all unstable/repelling, so real-world noise kicks you off them. This is exactly why chaotic systems look periodic-ish in the short run but never actually repeat.

---

## CLV — chaos only if retention is made endogenous

As stated, with `m`, `r`, `d` held constant:

$$CLV = m \cdot r/(1 - r + d)$$

this is just a fixed point — no dynamics, no chaos possible. It becomes a chaos candidate only if you feed margin back into retention:

$$r_{n+1} = h(m_n)$$

where reinvesting margin into loyalty programs increases next-cohort retention but with diminishing/negative returns at high investment (over-discounting cannibalizes margin, which starves the *next* round of retention spend). That reinvestment loop is again logistic-shaped:

$$r_{n+1} = \lambda \cdot r_n \cdot (1 - r_n)$$

and chaos appears the same way — past a bifurcation threshold, the retention rate no longer converges to one stable `r*`, but bounces unpredictably cohort to cohort.

---

## RFM — chaos in the segment-migration map

RFM is naturally a **vector-valued** dynamical system: fractions of customers in each segment (Champions, Loyal, Potential, At-Risk, Lost) migrate period-to-period based on how aggressively you incentivize each bucket:

$$v_{n+1} = M(incentive) \cdot v_n$$

where $v_n$ is the vector of segment population fractions at period `n`, and `M` is a (generally nonlinear, incentive-dependent) migration operator.

This is structurally a discrete competition/Markov-type map — multi-population discrete maps of this kind are known to exhibit chaos for certain incentive-elasticity parameter ranges (analogous to coupled logistic maps). Practically: if your win-back campaigns for "At-Risk" customers are too aggressive relative to how fast "Champions" churn down into that bucket, the segment proportions can start oscillating chaotically rather than settling into a stable customer-mix equilibrium — the RFM segment distribution could swing unpredictably quarter to quarter even under a fixed incentive *policy*.

---

## Bottom line

**FBR** is the model where this connection is tightest and most defensible mathematically — it already has an explicit price↔demand feedback loop built in, requiring no extra assumptions to become a genuine 1D chaotic map. **CLV** and **RFM** only connect to chaos once you explicitly make them iterative (a reinvestment loop for CLV, a segment-migration map for RFM) rather than treating them as one-shot formulas.

| | FBR | CLV | RFM |
|---|---|---|---|
| **State variable** | Traffic/revenue level, $N_n$ (scalar) | Retention rate $r_n$ (scalar, *only if* reinvestment-linked) | Vector: fraction of customers in each segment (Champion, Loyal, Potential, At-Risk, Lost) |

| | FBR | CLV | RFM |
|---|---|---|---|
| **Natural map form** | $N_{n+1} = \lambda N_n(1 - N_n/N_{\max})$ — logistic, closes the loop on its own (price ↔ demand) | Static formula by default (no dynamics); becomes $r_{n+1} = h(m_n)$ only if you explicitly model reinvestment feedback | Migration matrix M(incentive) applied each period to the segment-distribution vector — inherently coupled, multi-dimensional |
| **Control parameter (the "c" or "λ")** | Elasticity $E_d$ × fee-adjustment aggressiveness | Reinvestment intensity (how much margin gets plowed back into retention spend) | Incentive-intensity vector — one dial per segment, not a single number |
| **Does 3-property definition apply cleanly?** | Yes — it's a genuine 1D map, same family as $Q_c(x)=x^2+c$ | Only in the degenerate 1D reinvestment version; the plain CLV formula is a pure fixed point, chaos is structurally impossible | Not directly — dense periodic points / transitivity / sensitivity would need to be redefined for a vector-valued map; no 1D theorem covers this case |
| **Bifurcation route as parameter increases** | Period-doubling cascade, same qualitative shape as the orbit diagram in Fig. 8.1 — stable equilibrium → 2-cycle → 4-cycle → ... → chaos at the Feigenbaum threshold | None in the static form (always converges); in the reinvestment-loop version, same period-doubling route as FBR, since it reduces to the same logistic shape | Analogous concept exists (higher-dimensional bifurcations of the migration map), but period-doubling isn't guaranteed to be the route — multi-dimensional systems can bifurcate differently |
| **What "chaos" looks like in business terms** | Quarter-to-quarter overflight revenue swings wildly and unpredictably even though the pricing *policy* (the formula) never changed | Cohort retention rate no longer settles to a stable r* — bounces between values as reinvestment over/undershoots each cycle | Segment mix (% Champions vs. At-Risk) oscillates or drifts unpredictably even under a fixed incentive policy — RFM scores stop being a stable diagnostic |
| **Sensitive dependence — practical cost** | Two forecasts starting from fee rates that differ by a rounding error diverge completely within a handful of quarters (cf. Fig. 10.7's orbit table) | Same, but only once reinvestment feedback is added — otherwise the model is *immune* to sensitive dependence by construction | Small classification noise near segment boundaries can propagate into large differences in *which* incentive campaigns get triggered downstream |
| **Structural reason chaos can/ can't happen** | Nonlinearity is already built in (elasticity is nonlinear in R by construction) | Linear in its native form ($CLV = m.r/(1 - r + d)$ is monotonic, no folding) — chaos requires you to add a nonlinear feedback, it isn't there by default | Nonlinearity comes from competition between segments for the same finite customer pool — structurally similar to coupled logistic maps, but coupling means single-map tools underdetermine the behavior |
| **How you'd detect it empirically (analogue of the orbit diagram experiment)** | Sweep the fee-aggressiveness parameter, plot long-run traffic outcomes — look for the same period-doubling fan | Sweep reinvestment fraction, plot long-run retention rate | Requires a higher-dimensional analogue (basin/attractor plots per segment) — a single orbit diagram won't capture it |
| **Practical "chaos-avoidance" lever** | Cap the fee-adjustment step size (keeps λ below the bifurcation threshold, analogous to staying left of $c = -3/4$ in the orbit diagram) | Cap reinvestment-to-margin ratio | Stagger/dampen incentive rollout across segments rather than reacting to each period's RFM snapshot in full force |

| سوال کلی | پاسخ |
|---|---|
| آیا این مدل‌ها می‌توانند رفتار آشوبناک از خود نشان دهند؟ | بله، اما با شدت و ساختار متفاوت. مدل FBR بیشترین آمادگی را برای بروز آشوب دارد، CLV فقط در صورت ورود بازخوردهای غیرخطی می‌تواند آشوبناک شود، و RFM نیز به دلیل چندبعدی بودن، ظرفیت بروز رفتار پیچیده و شبه‌آشوبناک را دارد. |
| چه شرایطی برای بروز آشوب در این مدل‌ها لازم است؟ | وجود غیرخطی بودن، بازخورد پویا، تکرار زمانی و حساسیت به شرایط اولیه از شروط اصلی است. بدون این عوامل، مدل‌ها عمدتاً در حالت ایستا یا قابل پیش‌بینی باقی می‌مانند. |
| نقش پارامترهای کنترلی در پایداری یا ناپایداری سیستم چیست؟ | پارامترهای کنترلی تعیین می‌کنند که سیستم در ناحیه پایدار، تناوبی یا آشوبناک قرار گیرد. افزایش بیش از حد شدت این پارامترها می‌تواند سیستم را از تعادل خارج کرده و وارد ناحیه بی‌ثباتی کند. |
| آیا تغییرات کوچک در ورودی‌ها می‌تواند به تغییرات بزرگ در خروجی منجر شود؟ | بله. این ویژگی همان حساسیت به شرایط اولیه است. در FBR این اثر مستقیم‌تر دیده می‌شود، در CLV در نسخه بازخوردی ظاهر می‌شود، و در RFM به‌صورت تغییرات شدید در دسته‌بندی و سیاست‌های بازاریابی نمایان می‌گردد. |
| کدام مدل ذاتاً برای تحلیل آشوب مناسب‌تر است؟ | FBR مناسب‌ترین مدل برای تحلیل مستقیم آشوب است، زیرا ساختار آن ساده‌تر و رفتار غیرخطی آن مشخص‌تر است. CLV برای این منظور نیازمند توسعه دینامیکی است و RFM به تحلیل پیچیده‌تر چندمتغیره نیاز دارد. |
| چه تفاوتی میان مدل‌های تک‌بعدی و چندبعدی از نظر امکان بروز آشوب وجود دارد؟ | در مدل‌های تک‌بعدی مانند FBR و CLV، آشوب معمولاً از طریق نگاشت‌ها و نمودارهای دوشاخه‌ای بررسی می‌شود. در مدل چندبعدی RFM، آشوب پیچیده‌تر است و باید در فضای حالت چندمتغیره تحلیل شود. |
| برای تحلیل آشوب در این مدل‌ها از چه ابزارهای ریاضی می‌توان استفاده کرد؟ | برای FBR و CLV ابزارهایی مانند نمودار دوشاخه‌ای، فضای فاز و نمای لیاپانوف مناسب‌اند. برای RFM نیز علاوه بر این‌ها، ابزارهای تحلیل چندبعدی و بررسی بردار حالت لازم است. |

| سوال کلی | پاسخ |
|---|---|
| چگونه می‌توان رفتار آشوبناک را از رفتار تصادفی تشخیص داد؟ | رفتار آشوبناک با وجود ظاهر نامنظم، از یک ساختار قطعی غیرخطی پیروی می‌کند، در حالی که رفتار تصادفی فاقد چنین ساختاری است. تحلیل سری زمانی، فضای فاز و نمای لیاپانوف می‌تواند این تفاوت را آشکار کند. |
| آشوب چه اثری بر پیش‌بینی‌پذیری سیستم دارد؟ | آشوب باعث می‌شود پیش‌بینی بلندمدت سیستم بسیار دشوار یا حتی غیرممکن شود، زیرا اختلاف‌های کوچک اولیه در طول زمان به واگرایی شدید منجر می‌شوند. |
| شناخت آشوب چه کمکی به تصمیم‌گیری و کنترل سیستم می‌کند؟ | شناخت نواحی آشوب به مدیران کمک می‌کند از تنظیمات پرریسک اجتناب کنند، پارامترها را در محدوده پایدار نگه دارند و سیاست‌های کنترلی مناسب‌تری طراحی نمایند. |
| چه راه‌هایی برای جلوگیری یا کنترل ورود سیستم به ناحیه آشوب وجود دارد؟ | محدود کردن شدت تغییرات، کنترل بازخوردهای مثبت، کاهش اندازه گام تنظیمات، و اعمال سیاست‌های تدریجی از مهم‌ترین راهکارهای جلوگیری از ورود سیستم به آشوب هستند. |
| ارزش علمی و کاربردی تحلیل آشوب در این مدل‌ها چیست؟ | این تحلیل نشان می‌دهد که رفتار سیستم همیشه خطی و قابل پیش‌بینی نیست. از نظر علمی، مدل را عمیق‌تر و واقعی‌تر می‌کند و از نظر کاربردی، به بهبود پیش‌بینی، کنترل و تصمیم‌گیری در محیط‌های پیچیده کمک می‌نماید. |

| معیار تحلیل آشوب | FBR (کارمزد-محور) | CLV (ارزش طول عمر) | RFM (رفتار مشتری) |
|---|---|---|---|
| **نوع پویایی** | نگاشت غیرخطی مستقیم | بازخورد ایستا به پویا | سیستم دینامیکی برداری |
| **منبع اصلی آشوب** | کشش قیمت/کارمزد | نرخ بازسرمایه‌گذاری | تغییر وضعیت در فضای ویژگی‌ها |
| **پیچیدگی تحلیل** | تک‌بعدی (نمودار دوشاخه‌ای) | وابسته به متغیرهای زمانی | چندبعدی (فضای حالت پیچیده) |
| **پیش‌بینی‌پذیری** | بسیار حساس به λ | محدود به دوره بازخورد | به‌شدت ناپایدار در بلندمدت |
| **راهبرد کنترل** | سقف‌گذاری گام کارمزد | تعدیل نرخ جذب/نگهداشت | میراسازی انتقال دسته‌ها |

# منابع، داده‌ها و ابزارهای مورد استفاده

## منبع داده

- داده‌های مورد استفاده شامل مجموعه داده‌های طبقه‌بندی‌شده و گزارش‌شده از پروازهای عبوری از فضای هوایی ایران در بازه زمانی 2023 تا 2025 است که توسط شرکت فرودگاه ها در اختیار پروژه قرار گرفته است.

## منابع علمی


- Devaney, R. L. (2020). *A First Course in Chaotic Dynamical Systems: Theory and Experiment*. Chapman and Hall/CRC.


## ابزارهای نرم‌افزاری و تحلیلی

- Python 3.x — پیاده‌سازی الگوریتم‌ها و تحلیل داده‌ها
- JupyterLab — توسعه، مستندسازی و اجرای کدهای تحلیلی
- Pandas — پاک‌سازی، پردازش و تحلیل داده‌ها
- NumPy — محاسبات عددی و آرایه‌ای
- Matplotlib — تولید نمودارها و مصورسازی داده‌ها
- Seaborn — مصورسازی آماری و تحلیل اکتشافی داده‌ها
- Git — کنترل نسخه و مدیریت تغییرات پروژه
- GitHub — نگهداری کدها، مستندسازی و مدیریت مخزن پروژه

## ابزارهای هوش مصنوعی

- OpenAI ChatGPT — کمک در نگارش علمی، بازبینی متن، قالب‌بندی و مستندسازی
- Google NotebookLM — سازمان‌دهی منابع، یادداشت‌ها و استخراج نکات کلیدی
- Anthropic Claude — بازبینی نگارشی، پیشنهاد ساختار و بهبود بیان علمی

تمامی تحلیل‌ها، نتایج و تفسیرهای ارائه‌شده در این گزارش توسط نویسنده بررسی و تأیید شده‌اند. ابزارهای نرم‌افزاری و هوش مصنوعی صرفاً برای پردازش داده‌ها، تحلیل، مستندسازی و بهبود کیفیت نگارش مورد استفاده قرار گرفته‌اند.